

# Computer Graphics

Week 8: Lighting and Material

# Aims

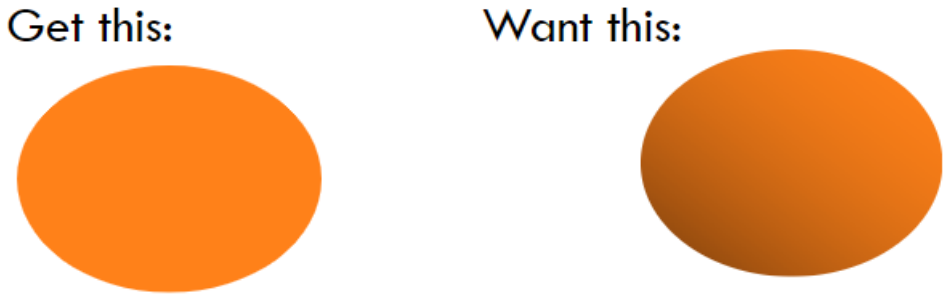
- To learn how to shade objects such that the resulting image looks more realistic
- To learn types of interactions between light sources and object surfaces
- To learn a simple reflection model (the Phong model) for a real time graphics hardware

# Sub Topics

- The Lighting Process
- Specular Reflection and Phong Shading
- Spotlight
- Sample Program

# Motivation

- Up to now, the color of object is defined explicitly by the source code, represented as RGB values.
- The explicit color values to define object's color cannot be used for producing more realistic images.



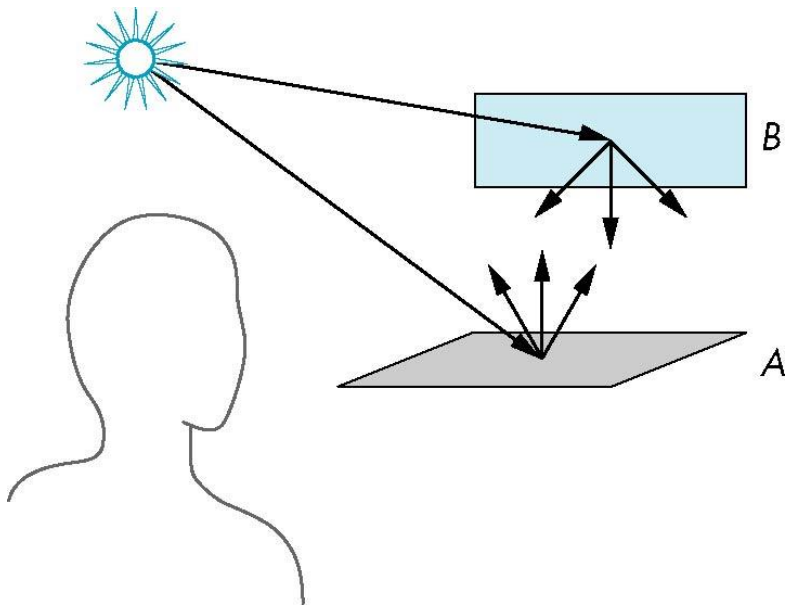
# Shading

- The interaction between light sources and object surfaces cause every points at the surface has different color (shaded differently)

- The shade can be influenced by the following factors
  - Light sources
  - Object's material
  - The viewer
  - The object's surface orientation

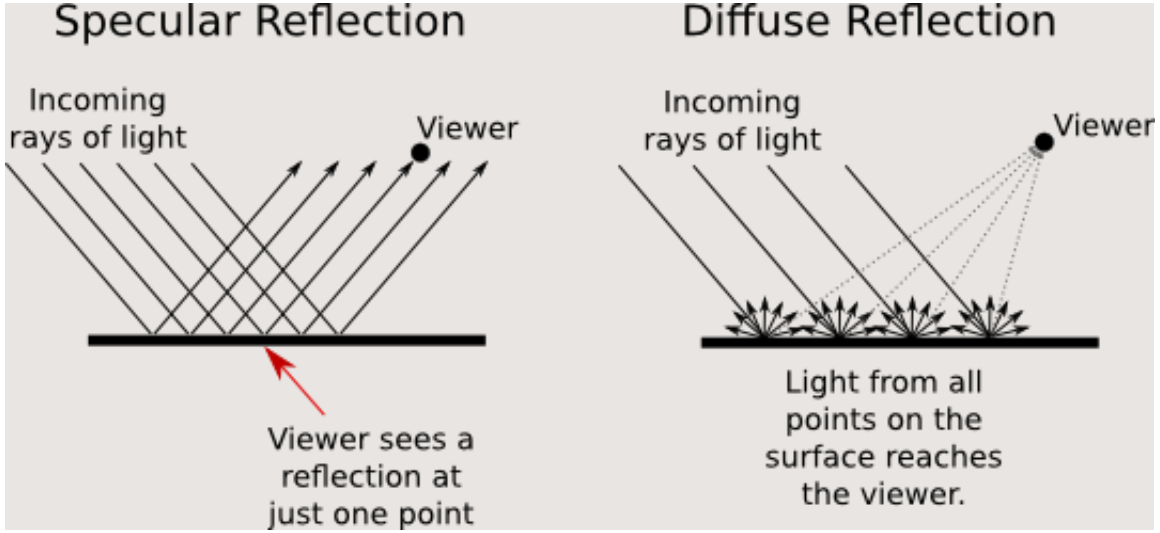
"In a 2-D photograph or painting, objects appear to be 3-D because we see small variations, or **shades**, in the colors of rendered objects." (OpenGL: A Primer)

# Lighting



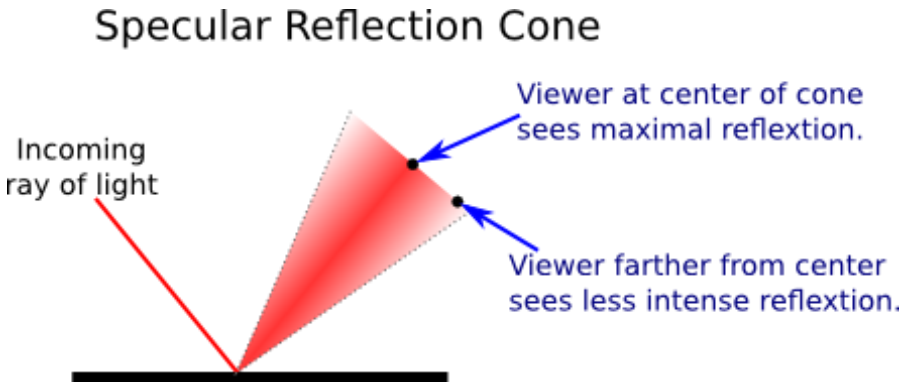

- **Goal**: to simulate light sources and its interactions with the object's surfaces
- **Material**: the object's surface property that defines the surface interaction with the light hit to it

# Types of Light Reflection in OpenGL

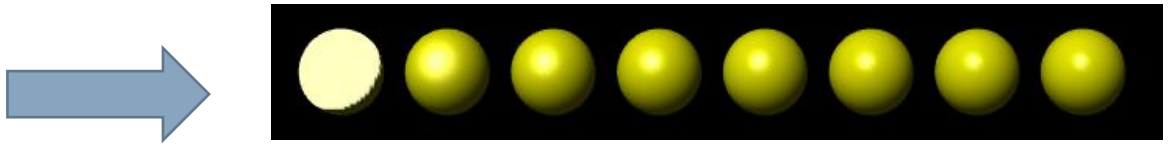


# Specular Reflection

- Shininess: the higher the shininess, the smaller and sharper the specular reflection cone



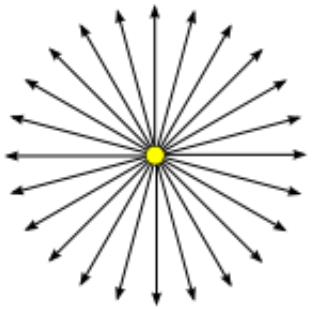

- The figure shows spheres ordered from the smallest shininess



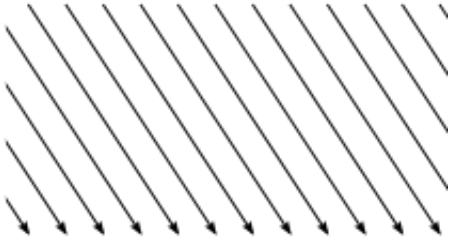
# Types of Material Color a.k.a. Reflectivity

- Diffuse color: the object's color
- Specular color: specular highlight color
- Ambient color: ambient light reflection color, usually has the same value as the diffuse color
- Emission color: generally, it's black (0, 0, 0), or there is no emission

# Light Source in OpenGL



POINT LIGHT
emits light in
all directions.



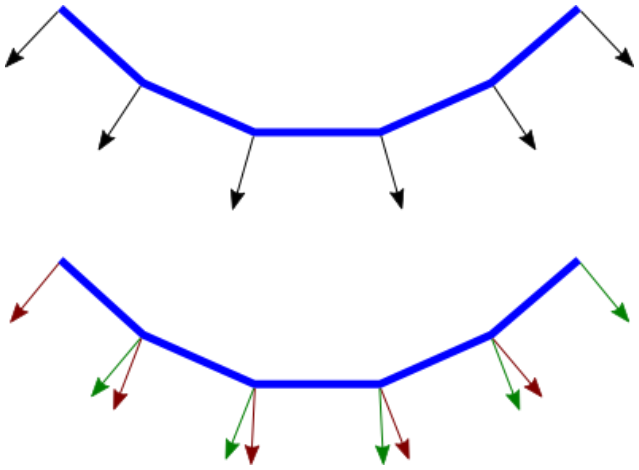
DIRECTIONAL LIGHT
has parallel light rays, all
from the same direction.

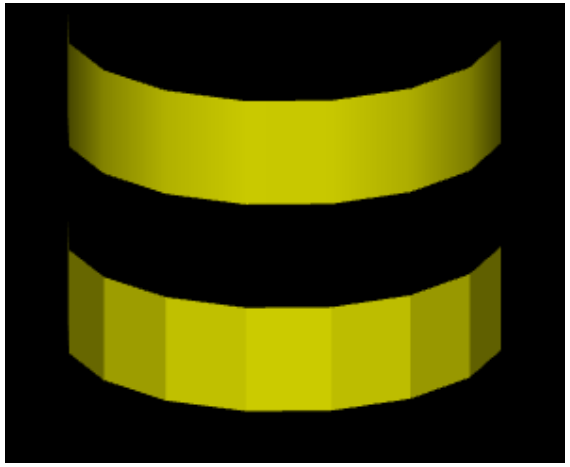
# Types of Light Source Colors a.k.a intensity or energy

- Diffuse light: the light source color that interacts with the material's diffuse color
- Specular light: the light source color that interacts with the material's specular color, usually has the same value as the diffuse light
- Ambient color: warna cahaya yang berinteraksi dengan warna ambient material

Real light sources do not have separate ambient, diffuse, and specular colors, and many computer graphics systems model light sources using just one color

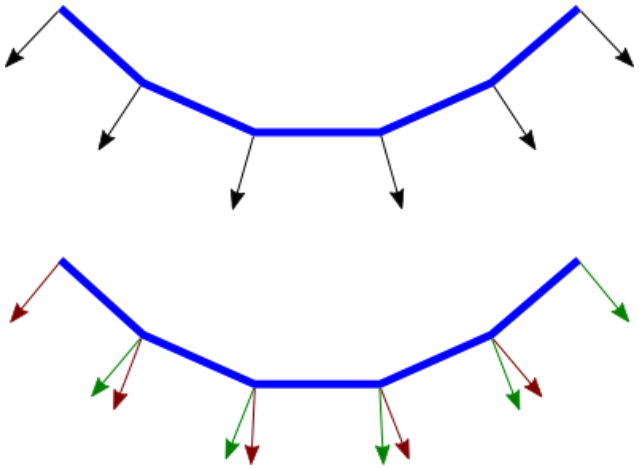
# Normal Vectors

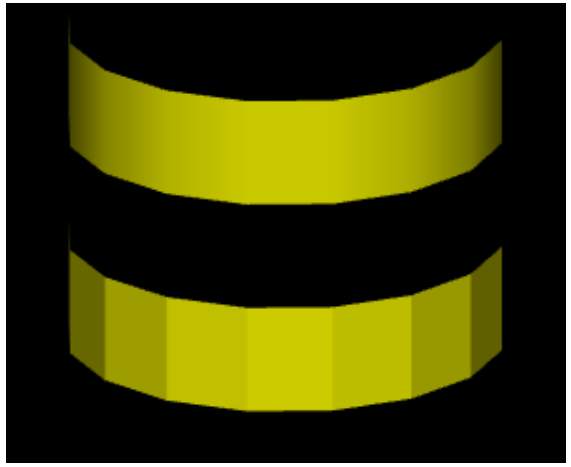




- A vector that is perpendicular to the surface
- A normal vector of length one is called a unit normal

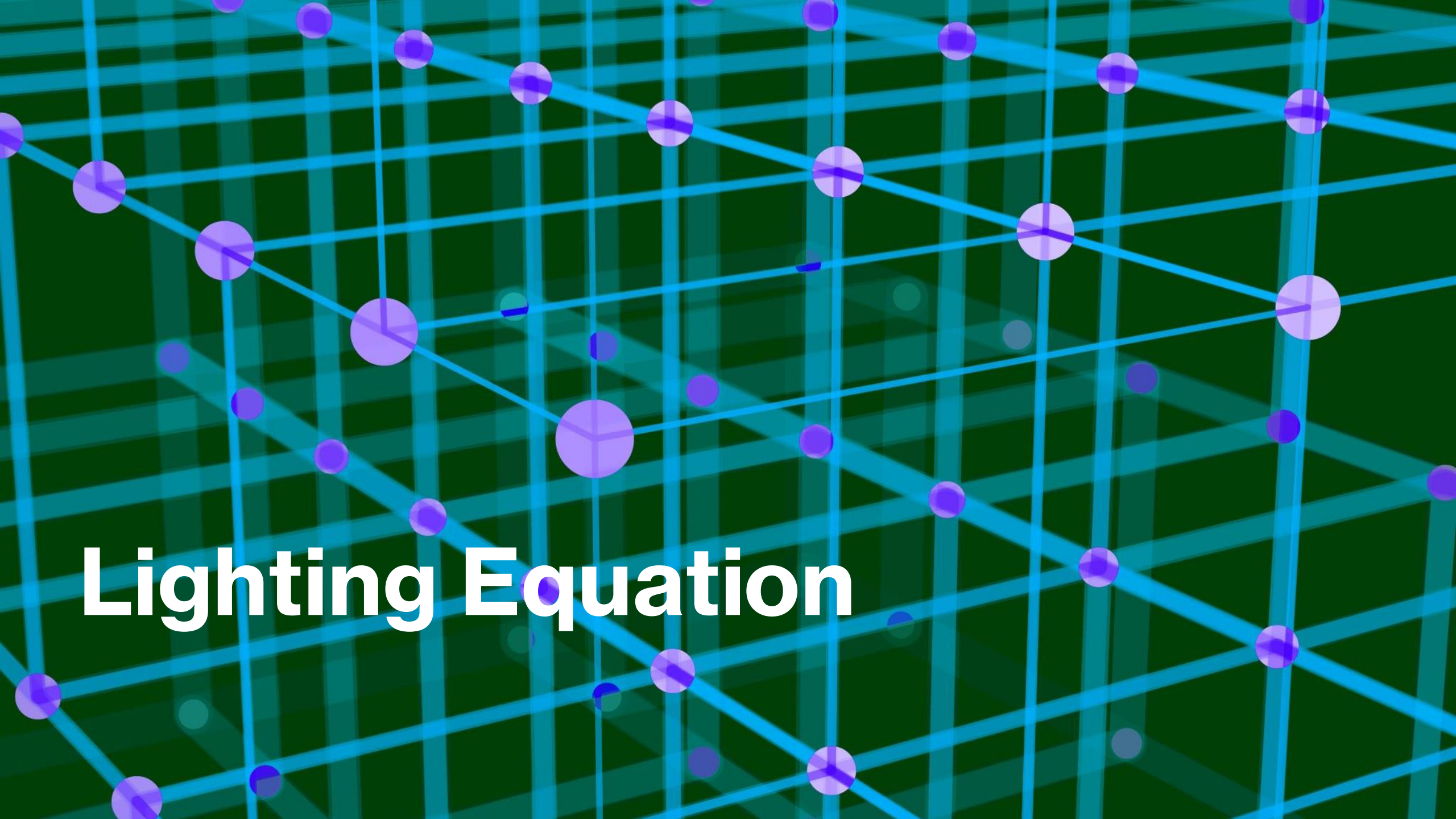
- For proper lighting calculations in OpenGL, a unit normal must be specified for each vertex.



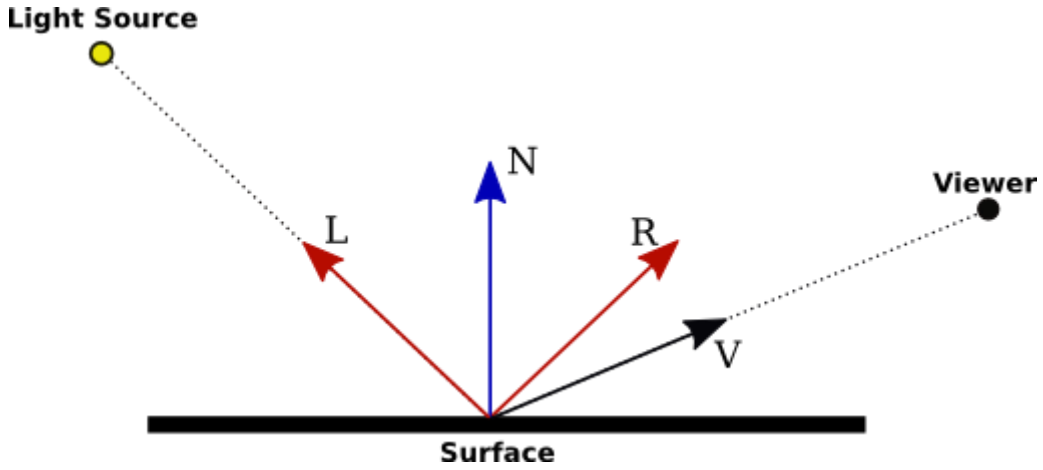


# Flat vs Smooth Shading

- Flat shading: makes a surface look like it is made of flat sides or facets.
- Smooth shading: makes it look more like a smooth surface
- The differences are the value of the normal vectors at each vertex


Lighting Equation

# Lighting Equation Factors




- N : the normal vector at the point whose color we want to compute
- L : a vector that points back along the direction from which the light arrives at the surface
- V : a vector that points in the direction of the viewer
- R : the direction of the reflected ray
- The angle between N and L is the same as the angle between N and R
- All of the vectors are unit vectors, with length 1
- Recall that for unit vectors A and B, the inner product A · B is equal to the cosine of the angle between the two vectors

# Lighting Equation

- Goal: to produce a color, (*r,g,b,a*), for a point on a surface
- The alpha (*a*) = the alpha component of the diffuse material color at that vertex
- Let's assume that the ambient, diffuse, specular, and emission colors of the material have RGB components ($ma_r$,$ma_g$,$ma_b$), ($md_r$,$md_g$,$md_b$), ($ms_r$,$ms_g$,$ms_b$), and ($me_r$,$me_g$,$me_b$), respectively
- The global ambient intensity is ($ga_r$,$ga_g$,$ga_b$).
- There can be several point and directional light sources, which we refer to as light number 0, light number 1, light number 2, and so on.

# Lighting Equation

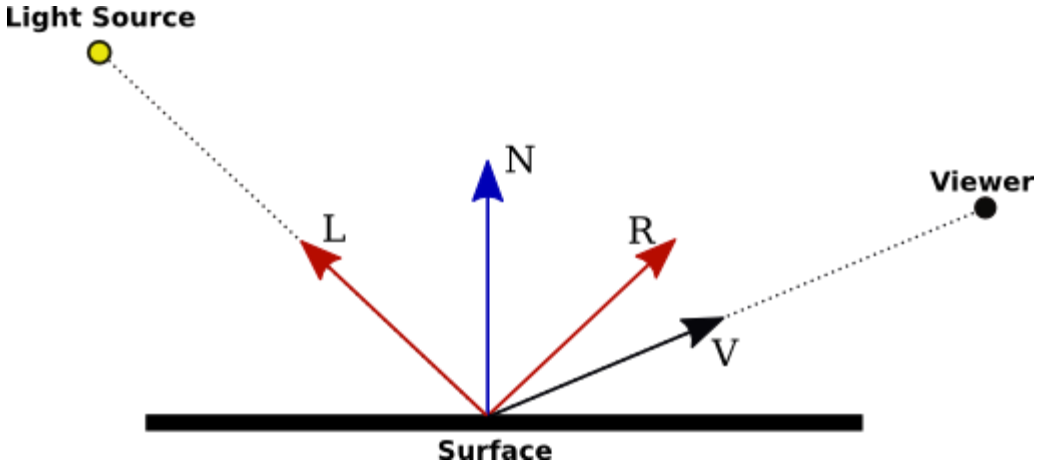
- Then, the red component of the vertex color will be:

$$r = me_r + ga_r * ma_r + I_{0,r} + I_{1,r} + I_{2,r} + \ldots$$

  where $I_{0,r}$ is the red component of the contribution to the color that comes from light number 0; $I_{1,r}$ is the contribution from light number 1; and so on.

- A similar equation holds for the green and blue components of the color.
- The emission color, $me_r$, is simply added to any other contributions to the color.
- And the contribution of global ambient light is obtained by multiplying the global ambient intensity, $ga_r$, by the material ambient color, $ma_r$.

# Lighting Equation




- Suppose that the **l**ight has **a**mbient, **d**iffuse, and **s**pecular color components ($la_r$,$la_g$,$la_b$), ($ld_r$,$ld_g$,$ld_b$), and ($ls_r$,$ls_g$,$ls_b$).
- Let *mh* be the value of the shininess property of the material.
- Then, the contribution of the light source to the red component of the vertex color can be computed as

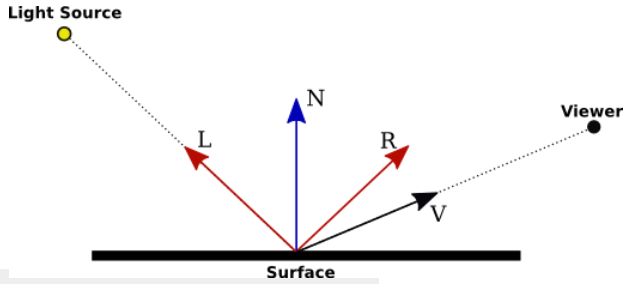
$$I_r = la_r * ma_r + f*( ld_r * md_r * (L \cdot N) + ls_r * ms_r * \max(0, V \cdot R)^{mh} )$$

with similar equations for the green and blue components. The value of f is 0 if the surface is facing away from the light and is 1 if the surface faces the light; that is, it accounts for the fact that the light only illuminates one side of the surface.



# Example 1

# Scenario




$$I_r = la_r * ma_r + f*( ld_r * md_r * (L \cdot N) + ls_r * ms_r * max(0, V \cdot R)^{mh} )$$

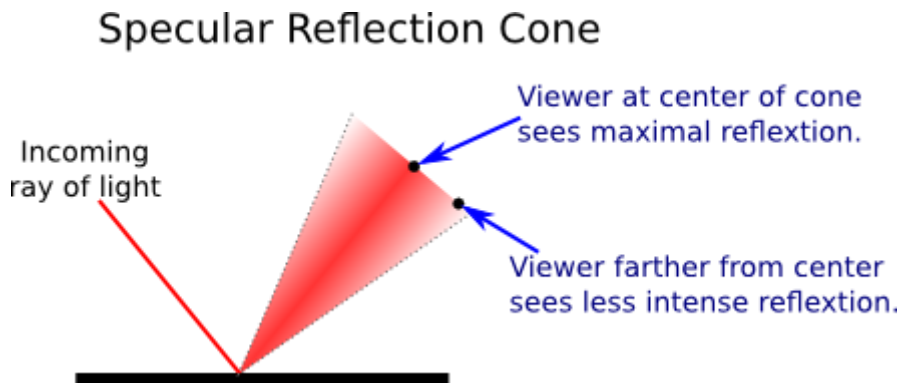
- A white light source from the direction of the camera (L = (0, 0, 1))
- There is only a diffuse reflection (no specular reflection)
  - Dot product (L, N) = dot product ((0, 0, 1), N) = N.z (the z-component of the normal vector N).
  - However, this assumes that N is also given in eye coordinates.
  - The normal vector will ordinarily come from the JavaScript side and will be expressed in object coordinates.
  - Before it is used in lighting calculations, it must be transformed to the eye coordinate system.

```
vec3 N = normalize( normalMatrix * normal );
```

- Material colors are as in the previous cube example

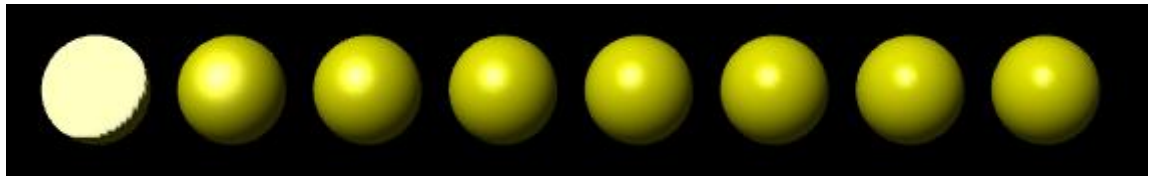
# Specular Reflection

- **Specular Highlights**: the reflection of the light source on the object's surface



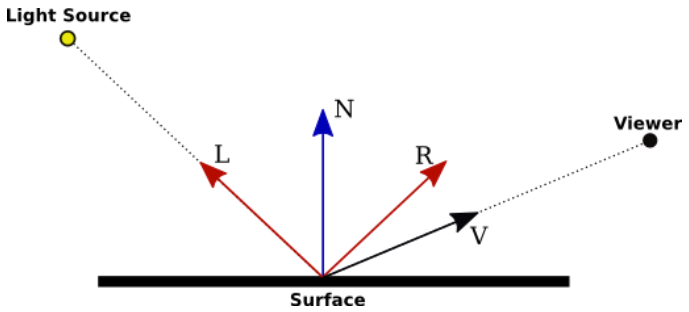
# Specular Reflection

- **Specular Highlights**: the reflection of the light source on the object's surface
- Specular reflection from a very shiny surface produces very narrow cones of reflected light; specular highlights on such a material are small and sharp
- In OpenGL, the material property that determines the size and sharpness of specular highlights is called **shininess** (a number in the range of 0 to 128): as the number increases, specular highlights get smaller.
- This image shows eight spheres that differ only in the value of the shininess material property (0, 16, 32, etc)



# Specular Lighting

$$I_r = la_r * ma_r + f*( ld_r * md_r * (L \cdot N) + ls_r * ms_r * max(0, V \cdot R)^{mh} )$$

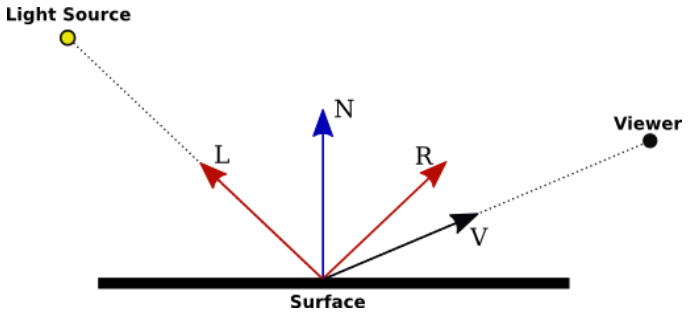



In perfect specular reflection, the viewer sees a specular highlight only if R is equal to V, which is very unlikely.

But in the lighting equation that we are using, the amount of specular reflection depends on the dot product R·V, which represents the cosine of the angle between R and V.

# Specular Lighting

$$I_r = la_r * ma_r + f*( ld_r * md_r * (L \cdot N) + ls_r * ms_r * \max(0, V \cdot R)^{mh} )$$




The unit vector R can be computed from L and N:
$2*(N \cdot L)*N - L$

GLSL has a built-in function reflect(I,N) that computes the reflection of a vector I through a unit normal vector N

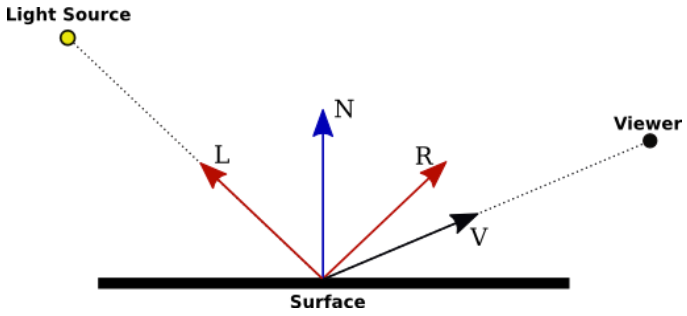
GLSL assumes a light direction vector that points from the light toward the surface, hence the output of *reflect(L,N) = -R*

Source code at the vertex shader:

```
R = -reflect(L,N);
```

# Specular Lighting

$$I_r = la_r * ma_r + f*( ld_r * md_r * (L \cdot N) + ls_r * ms_r * \max(0, V \cdot R)^{mh} )$$




The unit vector V points from the surface towards the position of the viewer.

For an orthographic projection, the viewer is essentially at infinite distance, and V = (0,0,1).

For a perspective projection, the viewer is at the point (0,0,0) in eye coordinates, and V = normalize(−eyeCoords) where eyeCoords contains the xyz coordinates of the surface point in the eye coordinate system

Source code di vertex shader:

```
V = normalize( -eyeCoords.xyz );
```

# More on Lighting

- Following the OpenGL convention, lightPosition is a vec4.
- For *directional light*
  - w = 0
  - the eye coordinates of the light: *lightPosition.xyz*
- Untuk *point light*
  - w ≠ 0
  - the eye coordinates of the light: *lightPosition.xyz/ lightPosition.w*

# Vertex Shader for Lighting Effect

- Look at the lighting-example-2.html

# Per-Vertex vs Per-Pixel Lighting

- Per-vertex lighting: perform the lighting computation in the vertex shader
- Per-pixel lighting: perform the lighting computation in the fragment shader
- Phong shading: per-pixel lighting with interpolated normal vectors

- An example of per-pixel lighting (Phong shading): lighting-example-3.html



# Next Topic

Texture Mapping